온라인 적응형 LLM 서빙의 인증 주체 간 제어 상태 오염

이도하*, 김민기*, 신가영*, 허영조*

*세종대학교 (학부생), *서강대학교 (학부생), *백석대학교 (졸업생), *아주대학교 (학부생)

Cross-Principal Control-State Contamination in Online-Adaptive LLM Serving: A vLLM Case Study and Feedback-Isolation Design Proposal

Do-Ha Lee*, Min-Gi Kim*, Ga-Yeong Shin*, Young-Jo Heo*

*Korea University(Graduate student)

요 약

대규모 언어 모델의 가중치는 추론 중 고정되지만 서빙 엔진은 실행 *feedback*으로 이후 정책을 조정한다. 본 논문은 한 인증 주체의 *feedback*이 공유 제어 상태를 거쳐 다른 주체의 후속 요청에 영향을 주는 '인증 주체 간 제어 상태 오염'을 정의한다. *vLLM*의 공격/대조×*estimator* 갱신 허용/차단 *2×2* 실험에서 처리량 *interaction*은 *-0.2478(95% CI [-0.2586,-0.2380], 8/8 adverse)*이었다. 또한 *feedback*을 문맥별 *shadow state*에 보관해 검증 후 *retire*하는 *CTFR*을 설계 제안으로 제시한다.

I. 서론

고성능 LLM 서빙 엔진은 모델 가중치와 별도로 변하는 runtime control state를 유지한다. Speculative decoding은 draft model이 제안한 토큰을 target model이 검증하여 직렬 추론 비용을 줄이며[1], 일부 시스템은 acceptance feedback으로 이후 검증량이나 draft policy를 온라인 조정한다[2]. 서로 신뢰하지 않는 요청이 하나의 controller state에 기여하면 request data plane과 장기 제어 상태 사이에 새로운 trust boundary가 생긴다.

본 연구는 이를 cross-principal adaptive control-state contamination으로 정의한다. 핵심은 동시 자원 경합이나 모델 학습 poisoning이 아니다. 주체 A의 요청이 종료된 뒤에도 A가 만든 feedback이 공유 상태에 남아 시간적으로 겹치지 않는 주체 B의 serving policy를 바꾸는지가 문제다. 본 논문의 기여는 ① 일반적인 상태 전이와 성립 조건 정의, ② 고정된 vLLM[3] 구성에서 estimator-update 개입을 통한 인과 검증, ③ feedback을 즉시 공유 상태에 반영하지 않는 CTFR 설계 제안이다. 모든 온라인 적응형 엔진이 취약하다고 주장하지 않는다.

II. 배경 및 문제 정의

2.1 온라인 적응형 검증

Draft 제안이 자주 채택되면 긴 speculation이 유리하지만 acceptance가 낮으면 검증 비용이 증가한다. Controller는 accept/reject 기록으로 acceptance를 추정하고 다음 요청의 verification budget을 조정할 수 있다. 이는 모델 가중치의 온라인 학습이 아니라 serving-control adaptation이다. Online Speculative Decoding은 실제 질의에 대한 지속적 적응의 성능 이점을 보였고[2], test-time adaptation 연구는 비신뢰 표본이 공유 적응 상태를 오염시킬 수 있음을

지적하였다[4]. 본 연구는 이 두 맥락 사이의 요청 주체→공유 controller→다른 주체의 후속 policy 경로를 시스템 보안 경계로 다룬다.

공유 상태를 s, 주체 p의 feedback과 문맥을 y_p, c_p, 후속 주체 q의 검증량을 b_q라 하면 다음과 같다.

$$s_{t+1}=F(s_t,y_p(t),\ c_p(t))$$
$$b_q(t+1)=\pi(s_{t+1},\ x_q(t+1)). \quad (1)$$

검사가 필요한 조건은 ① mutable state가 요청·주체 간 공유되고, ② 비신뢰 요청이 feedback에 영향을 주며, ③ estimator가 주체나 유효 policy 문맥을 구분하지 않고, ④ 변경 상태가 후속 자원 배분을 바꾸며, ⑤ 요청 종료 뒤에도 유지되고, ⑥ 주체별 격리나 기여량 제한이 없는 경우다.

2.2 취약 조건

다음 조건을 함께 만족하는 시스템은 인증 주체 간 제어 상태 오염의 잠재적인 공격 표면을 가진다.

첫째, 변경 가능한 controller state가 요청 또는 인증 주체 사이에서 공유된다.

둘째, 비신뢰 요청이 controller의 update에 사용되는 feedback에 영향을 줄 수 있다.

셋째, estimator가 feedback을 생성한 인증 주체, sampling policy 또는 workload context를 충분히 구분하지 않는다.

넷째, 변경된 state가 후속 요청의 verification budget이나 자원 배분을 변경한다.

다섯째, 공격 요청이 종료된 이후에도 state가 process 또는 worker에 유지된다.

여섯째, 주체별 상태 격리나 주체별 update contribution 제한이 존재하지 않는다.

이 문제는 model-weight poisoning, KV-cache contamination, prompt injection 및 일반적인 noisy-neighbor 문제와 구별된다. 공격자와 victim의 트래픽이 시간적으로 겹치지 않아도 효과가 지속될 수 있으며, 모델 가중치 자체는 변경되지 않기 때문이다.

2.3 위협 모델

공격자와 victim은 gateway가 인증한 서로 다른 principal이며, 하나의 vLLM worker를 합법적으로 공유한다고 가정한다. Gateway는 클라이언트가 전달한 외부 identity header를 제거하고 내부에서 인증된 principal 정보를 부여한다.

공격자는 정상 inference API를 통해 prompt, 지원되는 sampling parameter, seed, 요청 시점 및 요청 수를 선택할 수 있다. 그러나 host, 모델 가중치, GPU memory, estimator telemetry 또는 실험용 estimator-update gate에는 접근할 수 없다.

공격과 victim 측정을 분리하기 위해 공격 요청이 모두 완료되고 queue가 비워진 뒤 victim 요청을 수행한다. 따라서 측정한 성능 저하는 공격자와 victim의 동시 GPU 경합으로 설명할 수 없다.

본 실험에서 사용한 principal은 gateway가 구성한 실험적 trust boundary이다. 따라서 본 논문은 vLLM이 native multi-tenancy를 제공하거나 vLLM의 기존 tenant isolation이 우회되었다고 주장하지 않는다.

시험한 원인 경로는 다음과 같다.

요청별 target policy 변형

→ draft와 target 사이의 acceptance 저하

→ worker-shared OnlineAcceptanceEstimator 갱신

→ verification budget 변화

→ 후속 victim 요청의 처리량 및 지연 변화

보호 대상은 생성된 문장의 의미나 모델 가중치가 아니라, 후속 요청의 availability와 성능에 영향을 주는 mutable serving-controller state다.

3. vLLM 사례 실험

3.1 실험 환경

실험에는 다음 환경을 사용하였다.

vLLM commit: 4fe9e6f6e5647a159c9e65e4f3bf33dec5d12e1c

Target model: Qwen/Qwen3-4B

Draft model: z-lab/Qwen3-4B-DFlash-b16

Speculative tokens: 4개

GPU: NVIDIA L40S

AWS instance: g6e.2xlarge, Seoul Region

Topology: 단일 vLLM worker와 인증 gateway

외부 요청은 gateway를 통해서만 worker에 전달되도록 구성하였다. Gateway가 부여한 attacker 및 victim principal의 audit event를 worker가 처리한 요청과 1:1로 결합하여 identity, 요청 단계 및 처리 결과를 검증하였다.

3.2 실험 설계

주 확인 실험은 workload와 estimator update 여부를 교차한 2×2 factorial design이다.

P-S: placebo workload, estimator update 허용

P-B: placebo workload, estimator update 차단

A-S: attack workload, estimator update 허용

A-B: attack workload, estimator update 차단

Blocked arm에서도 speculative decoding, draft generation 및 target verification은 정상적으로 수행한다. 공격 phase에서 estimator의 feedback accumulation과 refit만 억제한다. 따라서 A-S와 A-B의 차이는 공격 workload나 GPU 연산의 존재가 아니라 attacker feedback이 공유 estimator에 반영되는지 여부다.

각 arm은 새로운 vLLM process에서 수행하였다. 하나의 arm은 다음 단계로 구성된다.

Victim warm-up 요청 1,024건

Victim pre window 128건씩 4회

Attacker intervention 요청 4,096건

Victim post window 128건씩 8회

네 arm의 실행 순서는 Williams-balanced order를 사용하였다. 총 8개의 temporal block을 수집했으며, 개별 요청을 독립 표본으로 간주하지 않고 네 arm으로 구성된 block을 통계적 추론 단위로 사용하였다.

주 효과량은 8개 post window의 pre-normalized throughput에 대한 다음 interaction이다.

$$I = [(A-S) - (A-B)] - [(P-S) - (P-B)] \quad (2)$$

신뢰구간은 paired-block bootstrap으로 계산하였다. 각 block의 효과 방향이 일관적인지는 exact sign test로 검증하였다.

모든 32개 arm이 완료되었으며, intervention 요청 실패와 gateway audit 누락은 발생하지 않았다.

3.3 Mechanism 결과

두 attack arm의 draft acceptance는 모두 0.0000이었다. 두 placebo arm의 draft acceptance는 모두 0.1875였다. 따라서 estimator update 여부와 관계없이 attack arm에는 동일한 low-acceptance verification stimulus가 전달되었다.

Estimator update를 차단한 P-B와 A-B에서는 serialized estimator state가 정확히 불변이었다. 반면 shared update를 허용한 조건에서는 estimator coefficient가 변경되었다.

Intercept[0]에 대한 attack-by-update interaction은 -13.8508이었다. 8개 post window의 평균 normalized verification-budget interaction은 -0.5163이었다. 이는 같은 공격 stimulus가 존재하더라도 estimator update의 허용 여부에 따라 후속 victim의 verification

policy가 달라졌음을 보여준다.

3.4 성능 결과

각 arm의 평균 post/pre 처리량은 다음과 같다.

A-S: 0.6847

A-B: 1.0018

P-S: 0.9269

P-B: 0.9962

Shared-minus-blocked 처리량 효과는 attack 조건에서 -0.3172, placebo 조건에서 -0.0693이었다. Placebo의 일반적인 update 효과를 차감한 주 interaction은 -0.2478이었다.

95% paired-block bootstrap 신뢰구간은 [-0.2586, -0.2380]이었다. 8개 block 모두 피해 방향의 interaction을 보였으며, one-sided exact sign test의 p-value는 0.00390625였다.

Latency interaction도 피해 방향으로 나타났다. Baseline-normalized p50 latency interaction은 +0.5053, p95 latency interaction은 +0.4788이었다.

처리량 interaction은 post_01부터 post_08까지 모든 victim window에서 음수였다. 따라서 공격 종료 후 arm당 1,024개의 victim 요청을 처리하는 동안 효과가 관측되었다.

다만 -0.2478은 baseline-normalized arm 사이의 24.78 percentage-point interaction이다. 이를 모든 운영 환경에서 발생하는 24.78%의 raw throughput 손실로 해석해서는 안 된다. 또한 본 실험은 8개 post window 이후에도 상태가 무기한 지속된다는 사실을 입증하지 않는다.

3.5 인과 해석

A-S와 A-B는 동일한 4,096건의 공격 요청, speculative decoding, target verification 및 GPU 작업을 수행하였다. 그러나 estimator update만 차단한 A-B에서는 estimator state와 후속 victim policy가 유지되었다.

또한 각 block에서 placebo의 shared-minus-blocked 차이를 측정해 차감하였다. 따라서 shared mode 자체의 일반적 효과나 시간에 따른 환경 변화만으로 결과를 설명하기 어렵다.

이 결과는 시험한 vLLM 구성에서 다음 인과 경로를 지지한다.

공격자가 조절한 verification feedback

→ worker-shared estimator update

→ 후속 victim의 verification policy 변화

→ 처리량 및 지연 성능 저하

4. 피드백 격리 설계 제안

4.1 설계 목표

Estimator update를 전부 비활성화하면 공격자의 feedback이 공유 state를 변경하는 것을 막을 수 있다. 그러나 정상 workload에 적응하여 얻을 수 있는 성능상의 이점도 사라진다.

Principal별 estimator를 완전히 분리하는 방법도 고려할 수 있다. 이 방식은 직접적인 cross-principal contamination을 차단하지만, principal 수에 따라 상태가 증가하며 여러 정상 principal로부터 수집한 유용한 feedback을 공유할 수 없다.

본 논문은 안전성과 적응성 사이의 trade-off를 명시적으로 다루기 위한 설계 방향으로 Context-Tagged Feedback Retirement를 제안한다.

4.2 Context-Tagged Feedback Retirement

CTFR은 CPU speculative execution의 shadow state 및 retire/squash 구조[5]와 optimistic concurrency control의 validate-before-commit 개념[6]에서 착안한다.

개별 기법 자체를 새로운 것으로 주장하지 않는다. 핵심은 비신뢰 LLM 요청에서 생성된 feedback을 serving-controller state에 즉시 반영하지 않고, 별도의 retirement boundary를 통해 공개하는 것이다.

CTFR은 현재 공개된 committed controller state C^v와 아직 공개되지 않은 shadow state U[(p, τ), v]를 구분한다.

p는 gateway가 인증한 principal이다.

τ는 서버가 생성한 policy-context tag다.

v는 feedback 생성 당시의 committed-state version이다.

Policy-context tag τ에는 target 및 draft model revision, estimator feature-contract version, 실제 적용된 sampling policy와 verification configuration이 포함된다. 클라이언트가 전달한 임의 header를 신뢰하지 않고, gateway와 worker가 인증 정보와 실제 적용 정책으로부터 context를 생성해야 한다.

CTFR의 요청 처리 과정은 다음과 같다.

첫째, gateway가 요청 principal을 인증하고 worker가 policy context τ를 계산한다.

둘째, 요청은 현재 committed state C^v만 읽는다.

셋째, 요청에서 발생한 acceptance feedback, sufficient statistics, observation count와 refit timing은 U[(p, τ), v]에만 기록한다.

넷째, 고정된 epoch에서 각 shadow state가 committed state에 대한 candidate update를 생성한다.

다섯째, retirement validator가 candidate를 검사한다.

여섯째, 모든 조건을 통과하면 quiescent barrier에서 새로운 committed version으로 반영한다. 조건을 통과하지 못하면 candidate를 squash하고 C^v를 그대로 유지한다.

4.3 Retirement 검증 조건

Retirement validator는 최소한 다음 조건을 검사해야 한다.

최소 observation 수

서로 다른 인증 principal로 구성된 quorum

Candidate update 사이의 방향 일치도

Principal별 최대 aggregate weight

Principal별 누적 influence credit

Coefficient update의 norm bound

고정 feature-grid에서의 최대 acceptance-probability 변화

오래된 committed version을 기반으로 생성된 stale feedback 여부

이 조건들은 feedback이 정상임을 증명하는 oracle이 아니다. CPU branch와 달리 arbitrary serving feedback에는 결과가 정상이라는 사실을 확정해 주는 branch-resolution event가 존재하지 않는다. 따라서 CTFR은 인증과 validator coverage, 영향력 제한 및 공격자가 통제할 수 있는 principal 수에 대한 가정 아래 update의 영향력을 제한한다.

4.4 보안 속성

CTFR의 첫 번째 보안 속성은 retire 전 committed-state invariance다.

C(after unretired feedback from p) = C(before feedback from p) (3)

즉, 특정 principal에서 생성된 feedback이 retire되기 전에는 unrelated request가 읽는 committed controller state가 정확히 동일해야 한다.

이를 만족하려면 coefficient만 분리해서는 안 된다. Observation count, pending sufficient statistics, refit counter와 refit timing처럼 이후 update 시점을 바꿀 수 있는 모든 adaptation-relevant state를 shadowing해야 한다.

Retirement 이후의 속성은 더 제한적이다. 성공적으로 retire된 candidate가 안전하다고 단정할 수 없으므로, principal별 weight, cumulative influence와 trust region을 통해 한 번의 commit과 누적 commit이 미칠 수 있는 영향을 제한해야 한다.

4.5 예비 구현 관찰과 주장 경계

단일 worker와 두 principal을 사용한 예비 prototype에서는 shared, frozen, tenant-local, ctfr-shadow 및 ctfr-full의 다섯 arm을 구성하였다. 다섯 arm은 총 12,960건의 요청을 오류 없이 완료하였다.

Ctfr-shadow에서는 retire되지 않은 feedback이 serialized committed state를 변경하지 않는 것을 확인하였다.

그러나 정상 feedback candidate의 최대 acceptance-probability 변화가 사전에 고정한 0.15 bound를 초과한 0.152207로 측정되었다. 이에 따라 ctfr-full의 retirement는 fail-closed 방식으로 거부되었으며 부분 commit은 발생하지 않았다.

이 결과는 shadow state와 명시적 commit boundary를 구현할 수 있으며, 검증 실패 시 committed state를 보존할 수 있음을 보여준다. 그러나 다음 사항은 입증하지 않는다.

CTFR이 실제 공격을 완화한다는 것

정상 workload에 대한 적응성을 보존한다는 것

Tenant-local estimator보다 우수하다는 것

Runtime overhead가 충분히 작다는 것

Concurrent request 및 multi-worker 환경에서 원자적이라는 것

따라서 본 논문에서 CTFR은 검증 완료된 완화책이 아니라, 실험 결과로부터 도출한 피드백 격리 설계 제안으로 한정한다.

5. 논의 및 한계

5.1 다른 서빙 엔진에 대한 일반화

본 연구가 일반화하는 것은 모든 서빙 엔진의 취약성이 아니라 검사 가능한 위험 구조다.

다른 엔진에 동일한 문제를 적용하려면 공유 mutable state, 비신뢰 feedback, principal 또는 policy context의 누락, 후속 policy와의 연결 및 request 종료 이후의 지속성을 각각 확인해야 한다.

두 번째 서빙 엔진에 대한 실증 없이 cross-engine vulnerability를 주장할 수는 없다. 따라서 본 연구는 vLLM을 실증된 사례 연구로 두고, 다른 시스템에는 분석 framework와 검사 조건만 제시한다.

5.2 완화 방법 비교

Principal별 estimator는 직접적인 cross-principal contamination을 방지하지만 상태 수가 증가하고 정상 feedback 공유의 이점을 잃을 수 있다.

Policy fingerprint별 routing은 서로 다른 sampling policy에서 생성된 feedback을 구분할 수 있지만, 동일 policy를 사용하는 서로 다른 principal의 악의적인 feedback을 제한하지 못할 수 있다.

Global clipping과 decay는 한 번의 update가 미치는 영향을 줄일 수 있지만, 공격자가 낮은 비율로 반복하는 slow-drip attack을 완전히 차단하지 못할 수 있다.

CTFR은 feedback의 origin과 visibility lifecycle을 명시한다는 장점이 있지만, validator의 정확성과 identity system에 의존하며 구현 복잡도가 증가한다.

5.3 연구의 한계

실험은 하나의 vLLM commit, 하나의 target 및 draft model pair, 하나의 GPU 종류, 단일 worker와 고정된 workload만을 대상으로 수행하였다.

Gateway principal은 실험적으로 구성한 trust boundary이며 vLLM의 native tenant 격리를 시험한 것이 아니다.

공격 종료 후 8개 post window까지만 측정하였으므로 전체 recovery half-life와 무기한 persistence는 확인하지 않았다.

공격자가 사용한 GPU 시간 대비 victim의 누적 피해가 경제적으로 증폭되는지도 확립하지 않았다.

다른 model pair, concurrency, 정상 workload 분포 및 다른 LLM serving engine에서의 재현성은 후속 실험이 필요하다.

CTFR prototype은 quiescent barrier만 사용하였다. Concurrent version pinning, stale feedback 처리, crash atomicity, multi-worker publication, 동적 principal allocation 및 eviction은 구현하거나 평가하지 않았다.

6. 결론

온라인 적응형 LLM 서빙은 고정된 모델 가중치와 별도로 변경 가능한 control-state trust boundary를 만든다.

본 연구는 한 인증 주체의 요청 feedback이 공유 serving-controller state를 변경하고 다른 인증 주체의 후속 처리 정책에 영향을 주는 인증 주체 간 제어 상태 오염 문제를 정의하였다.

고정된 vLLM adaptive-verification 구성에서 공격 및 대조 workload와 estimator update 허용 및 차단을 교차한 2×2 factorial 실험을 수행하였다. 그 결과, 공격에 특이적인 후속 victim의 처리량 및 지연 변화가 worker-shared estimator update를 필요로 한다는 인과적 증거를 확보하였다.

또한 비신뢰 feedback을 인증 주체 및 policy context별 shadow state에 보관하고, 검증된 candidate만 retire하는 CTFR을 설계 방향으로 제안하였다. CTFR은 현재 검증된 방어책이 아니며, 후속 연구에서는 정상 적응성 보존, 공격 완화 효과, overhead, concurrency 및 multi-worker 환경에서의 원자성을 검증해야 한다.

참고문헌

[1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” Proceedings of the 40th International Conference on Machine Learning, pp. 19274-19286, 2023.

[2] X. Liu, L. Hu, P. Bailis, A. Cheung, Z. Deng, I. Stoica, and H. Zhang, “Online Speculative Decoding,” Proceedings of the 41st International Conference on Machine Learning, pp. 31131-31146, 2024.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” Proceedings of the 29th ACM Symposium on Operating Systems Principles, 2023.

[4] T. Wu, F. Jia, X. Qi, J. T. Wang, V. Sehwag, S. Mahloujifar, and P. Mittal, “Uncovering Adversarial Risks of Test-Time Adaptation,” Proceedings of the 40th International Conference on Machine Learning, pp. 37456-37495, 2023.

[5] K. N. Khasawneh, E. M. Koruyeh, C. Song, D. Evtyushkin, D. Ponomarev, and N. Abu-Ghazaleh, “SafeSpec: Banishing the Spectre of a Meltdown with Leakage-Free Speculation,” Proceedings of the 56th ACM/IEEE Design Automation Conference, 2019.

[6] H. T. Kung and J. T. Robinson, “On Optimistic Methods for Concurrency Control,” ACM Transactions on Database Systems, vol. 6, no. 2, pp. 213-226, 1981.

[참고문헌]

[1] D.S.Milojicic, V.Kalogeraki, R.Lukose,K.Nagaraja, J.Pruyne, B.Richard, S.Rollins and Z.Xu, Peer to Peer Computing, HP Laboratories Palo Alto HPL-2002-57, March, 2002.